

The GitHub Copilot harness

Shaping what Copilot knows, can do, and may do

- Nine layers around the model: **instructions**, prompts, MCP, skills, agents, plugins, memory, hooks, interop
- Each layer answers one question: what Copilot knows, what it can do, or what it may do
- The goal: the **right context at the right moment**, and nothing more

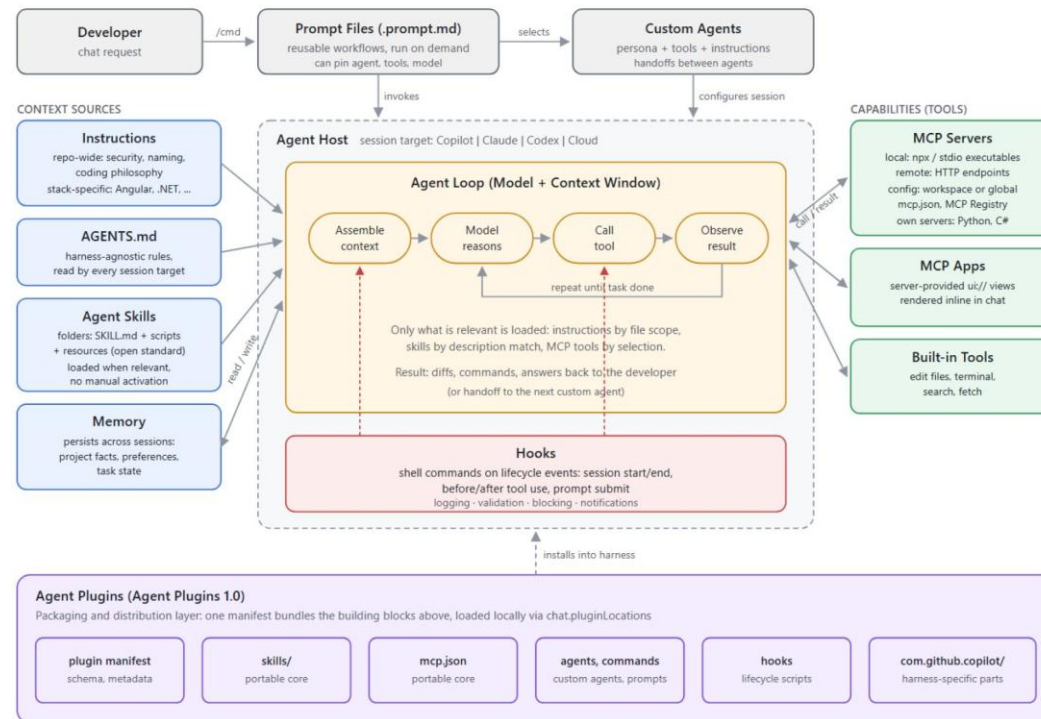


How the customization layers work together

One agent loop, fed by context sources and capabilities

- **Entry points:** the developer's chat request, a / prompt file, or a selected custom agent
- **Agent loop:** assemble context, reason, call a tool, observe, repeat until done
- **Context sources:** instructions, AGENTS.md, skills, memory
- **Capabilities and guards:** MCP servers and apps; hooks on lifecycle events

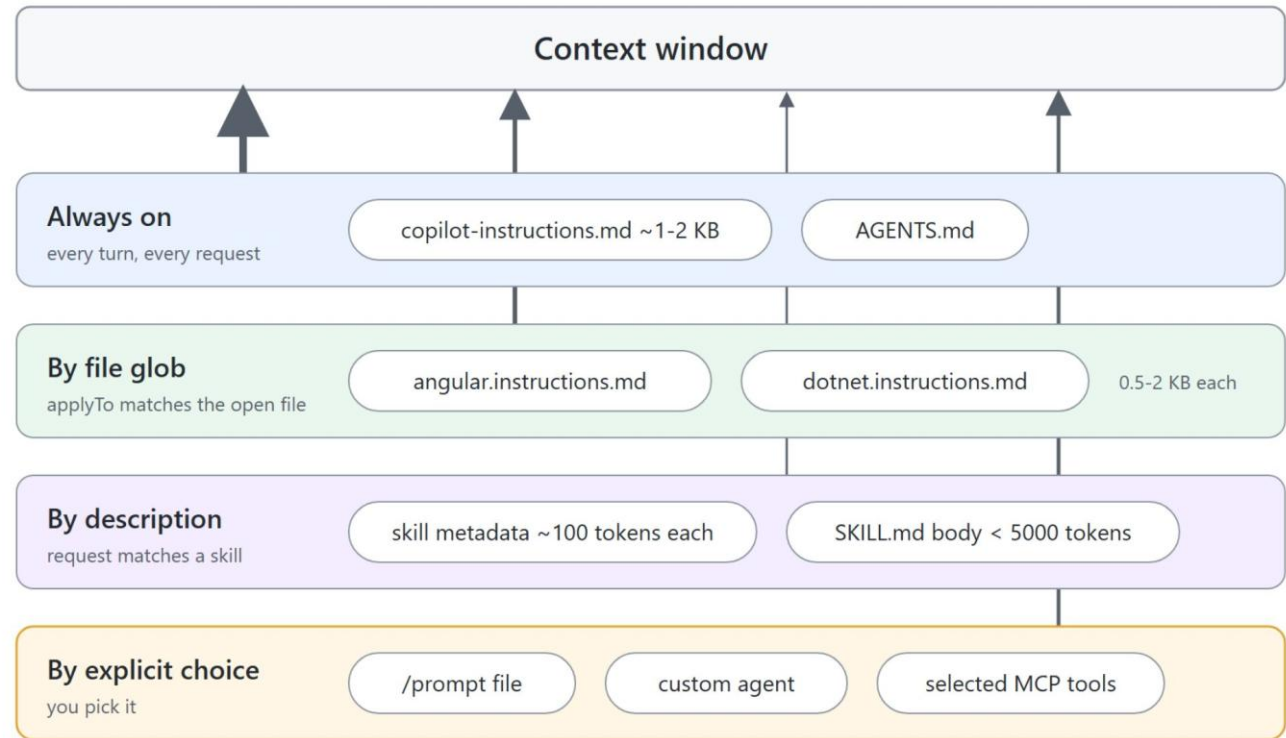
Agentic Harness: How the Customization Layers Work Together



Context is the budget

Every layer loads on a different trigger

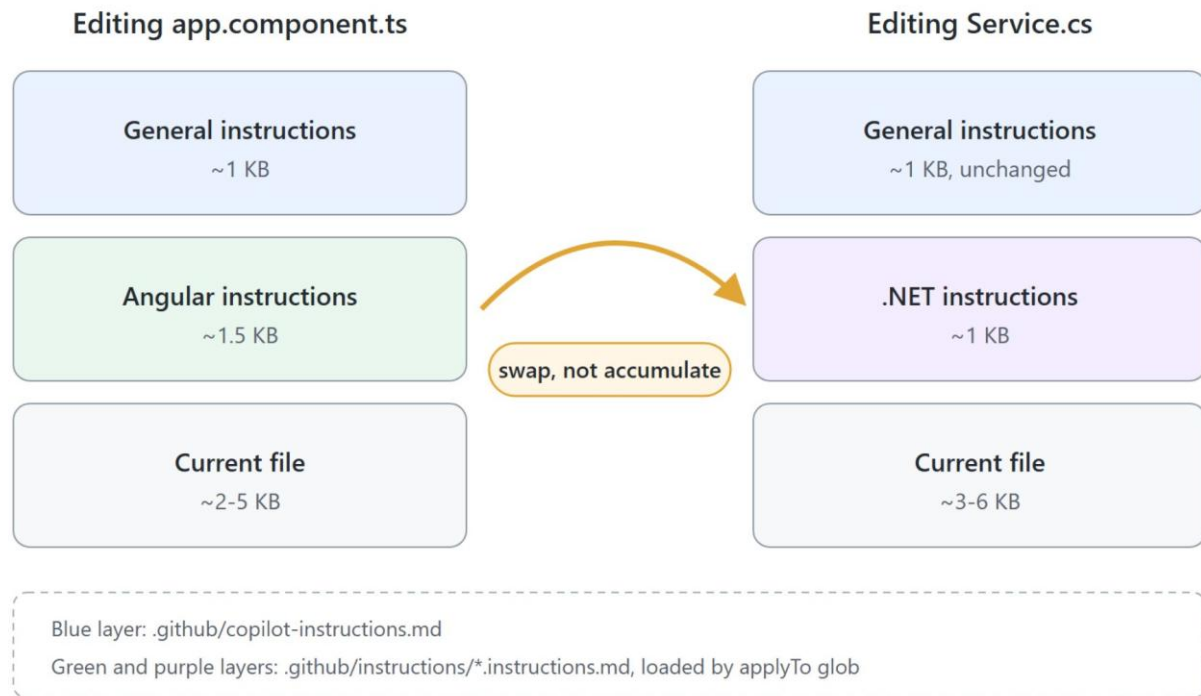
- **Always on:** repository-wide instructions and AGENTS.md, reloaded on every turn
- **By file glob:** stack instructions activate only for matching files
- **By description:** skills cost about 100 tokens each until a request matches
- **By explicit choice:** prompt files, custom agents, selected MCP tools



Instructions: always on, or scoped by file

General rules plus stack rules that swap in and out

- `.github/copilot-instructions.md`: security, naming, coding philosophy, loaded every turn
- `.github/instructions/*.instructions.md`: one file per stack, activated by an `applyTo` glob
- Opening a .NET file swaps the Angular rules out instead of stacking both
- Enable with `chat.instructionsFilesLocations`



Prompt files: workflows on demand

A request you package once and fire with a slash

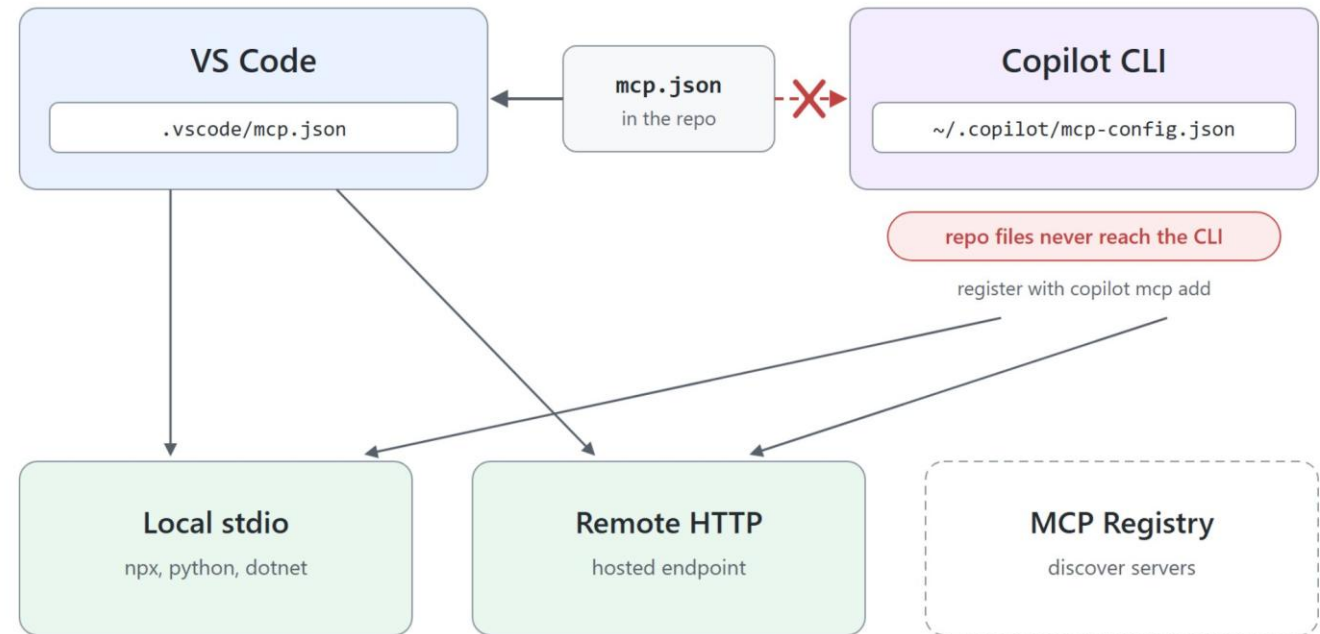
- `.prompt.md` files in `.github/prompts/`, run by typing `/name` in chat
- Frontmatter can pin an agent, tools, and a model
- All documentation in this class repo was produced with `/describe-module`
- VS Code 1.129 **migrates prompt files to skills**
(`chat.customizations.promptMigration.enabled`)



MCP: tools beyond the editor

Local or remote servers, and who reads which config

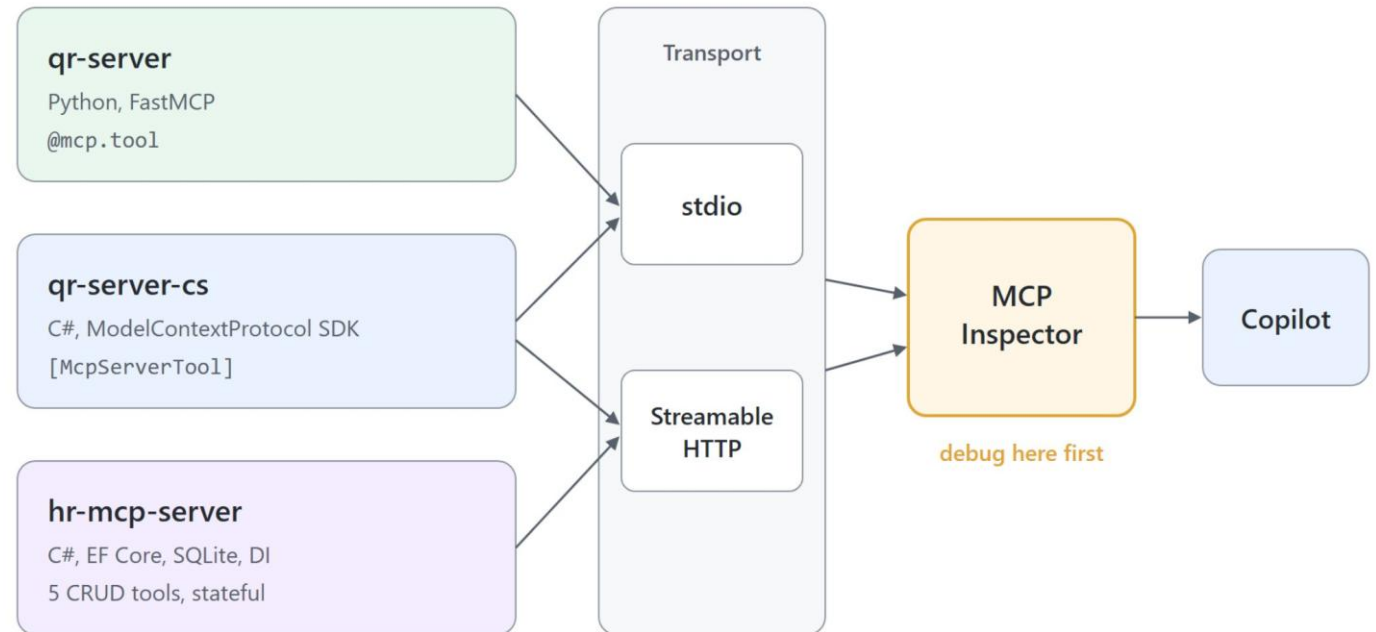
- MCP connects Copilot to **external tools and data**: browsers, docs, business systems
- Servers run **local** (stdio executables) or **remote** (HTTP endpoints)
- VS Code reads `.vscode/mcp.json`, top-level key `servers`
- The CLI reads only `~/.copilot/mcp-config.json`, written by `copilot mcp add`



Build your own MCP server

Python or C#, one contract, tested before any client sees it

- **Python:** FastMCP, one decorator turns a function into a tool
- **C#:** the same tool contract expressed with attributes
- **State:** `hr-mcp-server` adds dependency injection, EF Core, and five CRUD tools
- Pick **stdio** or **Streamable HTTP**, then debug in the **MCP Inspector** first



MCP Apps: UI inside the chat

A tool result that renders instead of printing

- The server declares a **ui:// resource** next to its tool
- The chat renders it as an **interactive UI**: charts, forms, QR codes
- Demo: a Python FastMCP server that draws its QR code in Copilot Chat

generate a qr code for <https://www.linkedin.com/in/alexander-kastil/>

> Used 2 references

The MCP servers `azure|devops`, `ash|-mcp`, `gith|` may have new tools and require interaction to start. Start them now?

✓ Tried to generate QR code for LinkedIn URL but was blocked

✓ Ran `generate_qr` - qr-code (MCP Server)

file.png



Add Context...

Describe what to build next

Skills: knowledge that loads itself

Progressive disclosure in three levels

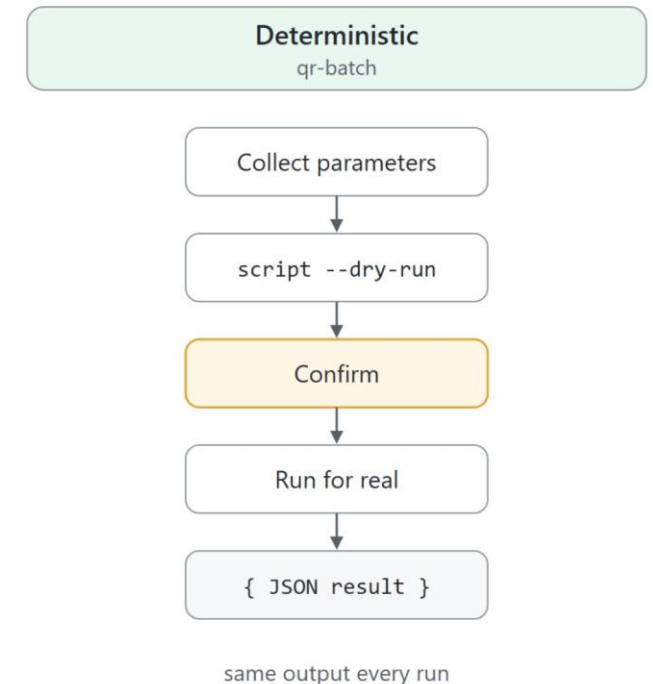
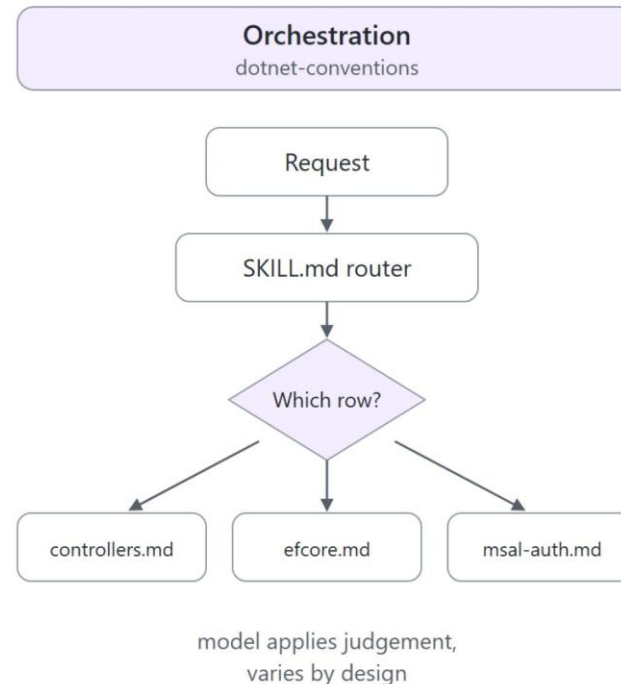
- A folder with **SKILL.md** plus optional scripts, references, and assets
- Open **Agent Skills** standard: Copilot CLI, coding agent, code review, VS Code, Claude Code
- Only name and description load at startup; the body loads on a match
- The **description** is the trigger: name the user's intent, not the implementation



Orchestration or deterministic

Judgement from the model, or output from a script

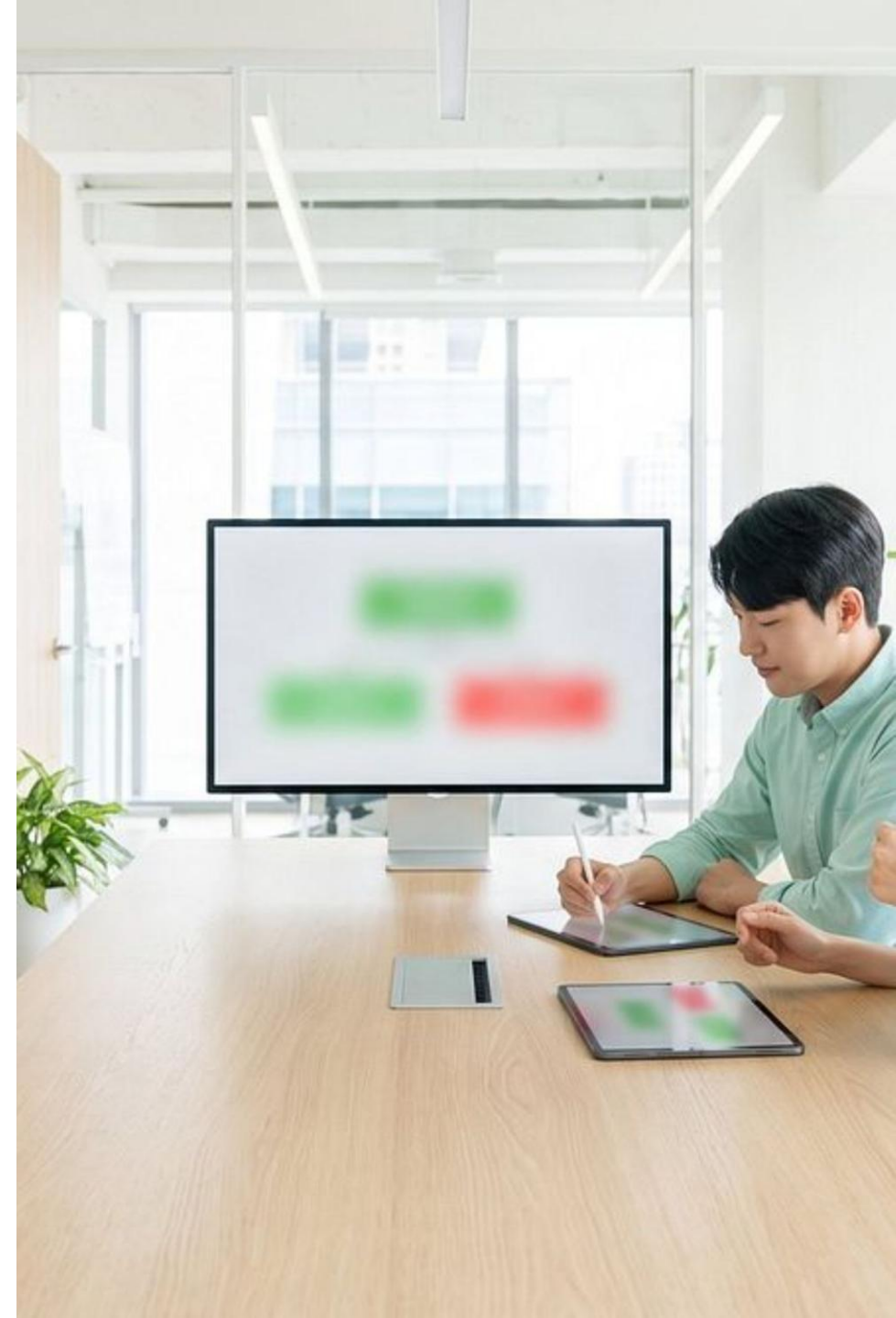
- **Orchestration:** `SKILL.md` holds the procedure, the model does the work
- **Deterministic:** the model collects parameters, a bundled script produces the output
- Bundle a script when output must be exact or the model cannot produce it (PNG, PPTX, QR code)

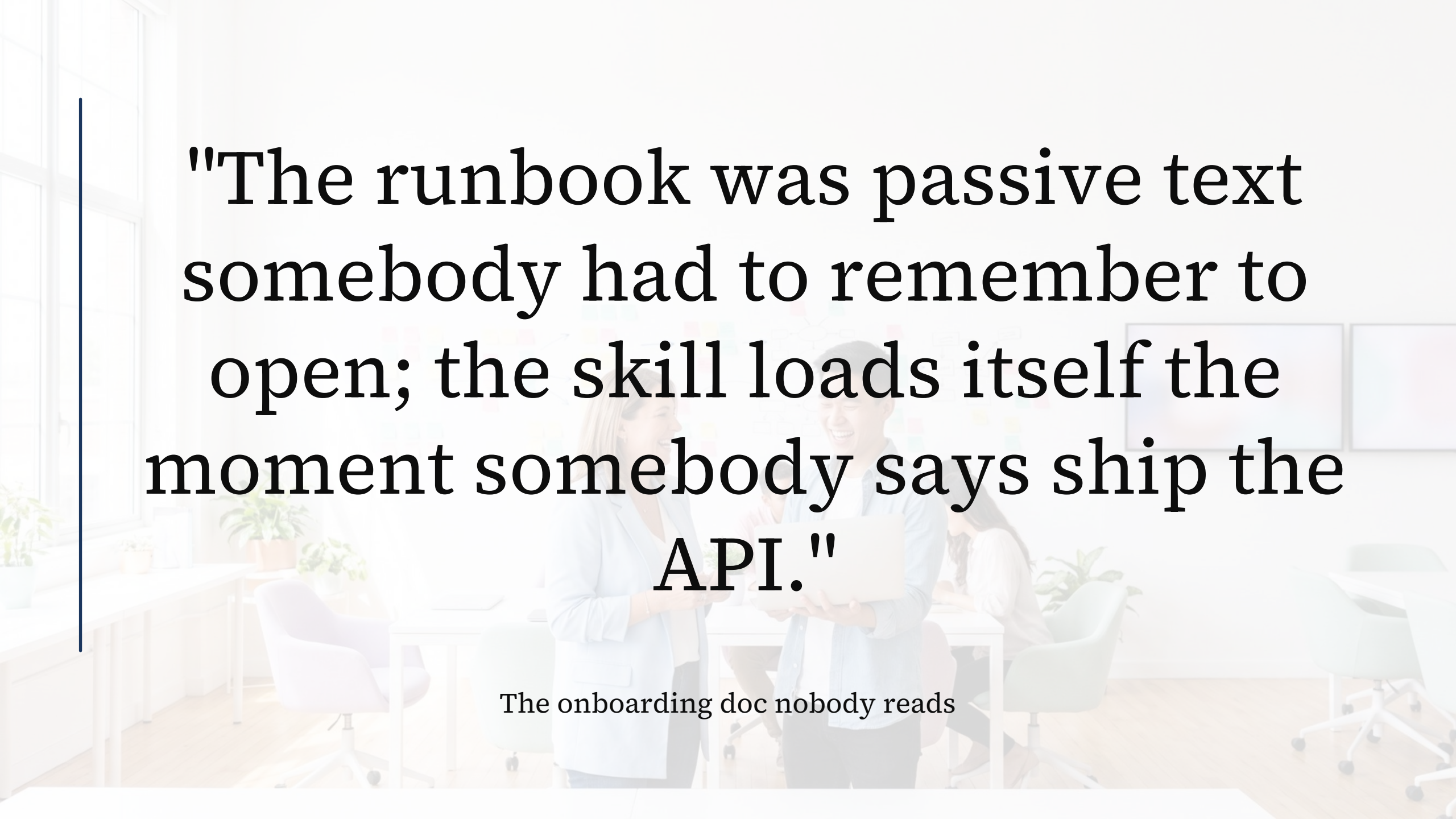


Test the trigger, then the output

Two failure modes, two tests, in that order

- **Trigger:** about 20 prompts, half should load, half are **near-misses** that must not
- Run each prompt three times; a positive passes above a 0.5 trigger rate
- Tune the description on 60 percent, pick the best score on the other 40
- **Output:** run each eval with and without the skill and read the **delta**





"The runbook was passive text
somebody had to remember to
open; the skill loads itself the
moment somebody says ship the
API."

The onboarding doc nobody reads

Custom agents and handoffs

Personas with their own tools, handing over the task

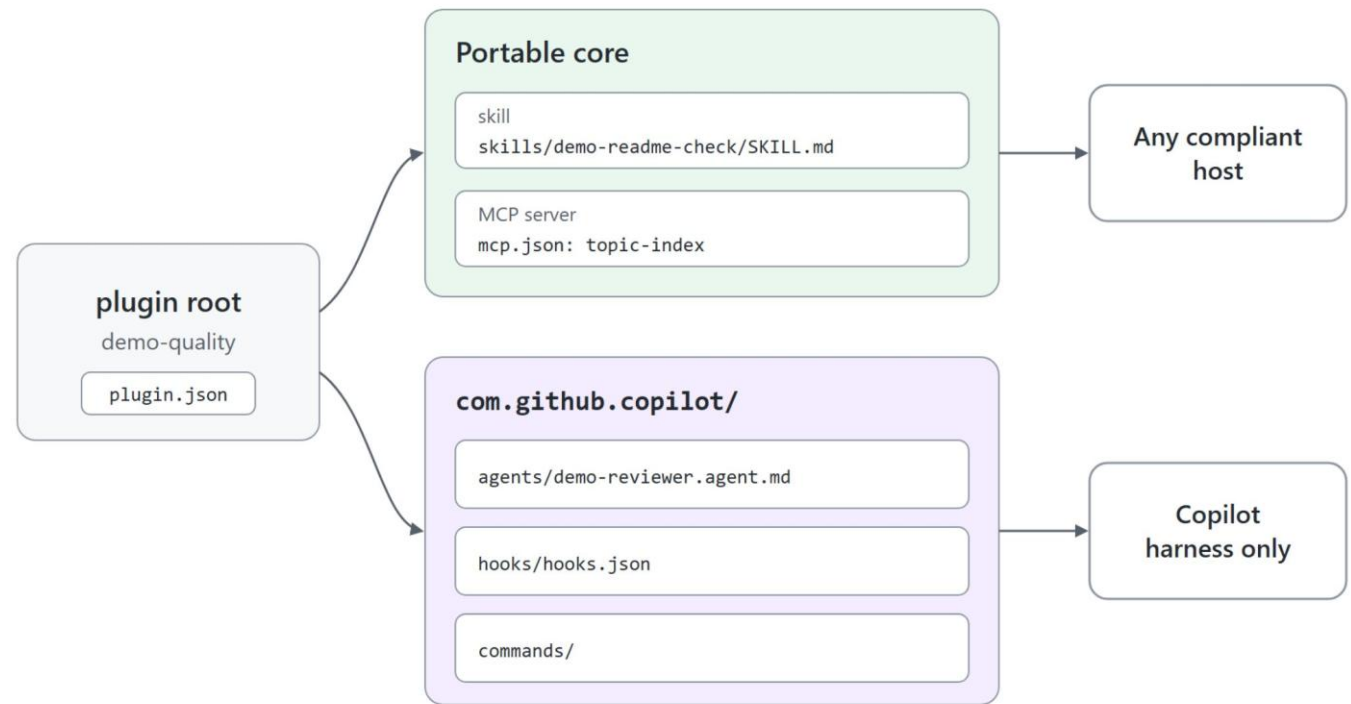
- `.agent.md` files in `.github/agents/`: persona, instructions, tool list, model
- **Handoffs** in frontmatter: `label`, `agent`, `prompt`, optional `send` and `model`
- A handoff button moves the session on: **plan, implement, review**
- This class ships planner, coder, frontend, orchestrator, GitHub Actions, Terraform, and more



Plugins: one bundle, a portable core

Agent Plugins 1.0 is an open standard

- One plugin bundles **skills, MCP servers, agents, hooks, and slash commands**
- **Portable:** `skills/` and `mcp.json`, understood by any compliant host
- **Copilot only:** agents, hooks, commands under `com.github.copilot/`
- Sample `demo-quality` ships a skill, an MCP server, an agent, and a hook



Discovered by convention, not declared

The manifest names the plugin, the folders name the parts

- `plugin.json` requires only `$schema` and `name`; extra keys are rejected
- No `skills` or `mcpServers` key: a file in the right folder is found
- VS Code: `chat.pluginLocations`; CLI: `copilot --plugin-dir <path> skill list`

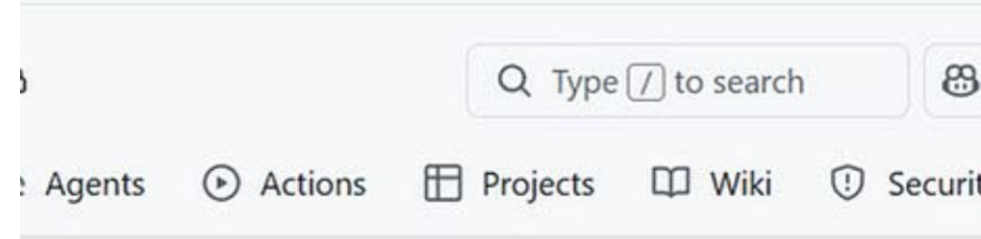
```
{
  "$schema": "https://agent-
plugins.org/schemas/1.0.0/plugin.schema.json",
  "name": "demo-quality"
}
```



Copilot Memory

Repository facts Copilot learns and keeps

- **Repository-scoped** facts Copilot deduces, such as "deployment config always from deploy.json"
- Validated against **code citations**; deleted after **28 days** unused
- Used by the coding agent, code review, and the Copilot CLI
- Off by default: `github.copilot.chat.copilotMemory.enabled`



Copilot memory

☐ 3 memories

☐ Do not restore deleted files, folders, or other items
File deletion handling

☐ When deploying to Azure, always retrieve deployment configuration from .github/deploy.json instead of hardcoding values
Deployment configuration

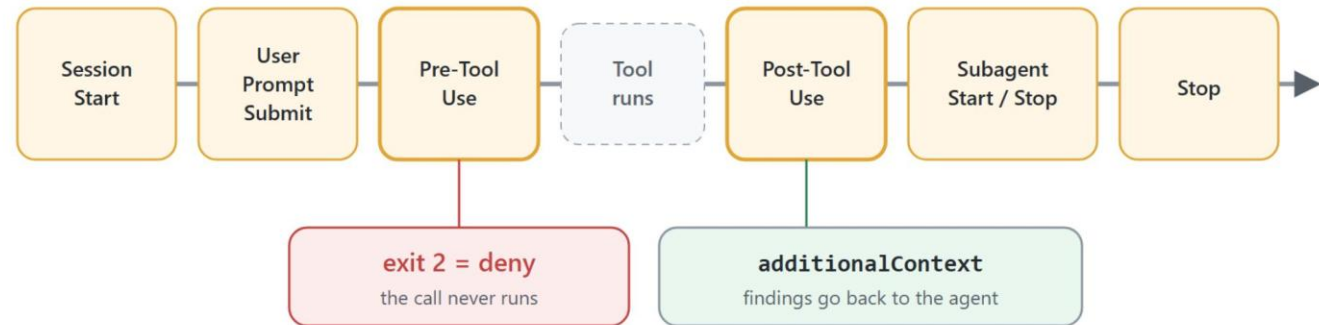
☐ Use camelCase for private variables in C#.
C# naming conventions

Hooks: policy at lifecycle events

Shell commands the agent cannot skip

- Configured in `.github/hooks/hooks.json`, scoped with a `matcher` regex
- `preToolUse` exit code 2 denies the call outright
- `postToolUse` returns `additionalContext: instructions-guard.ps1` lints the instructions file and hands findings back

`.github/hooks/hooks.json`: a `matcher` regex scopes each hook to the tools it cares about



logging · validation · blocking · notifications

Three gates between agent and machine

Sandbox, assisted approvals, hooks

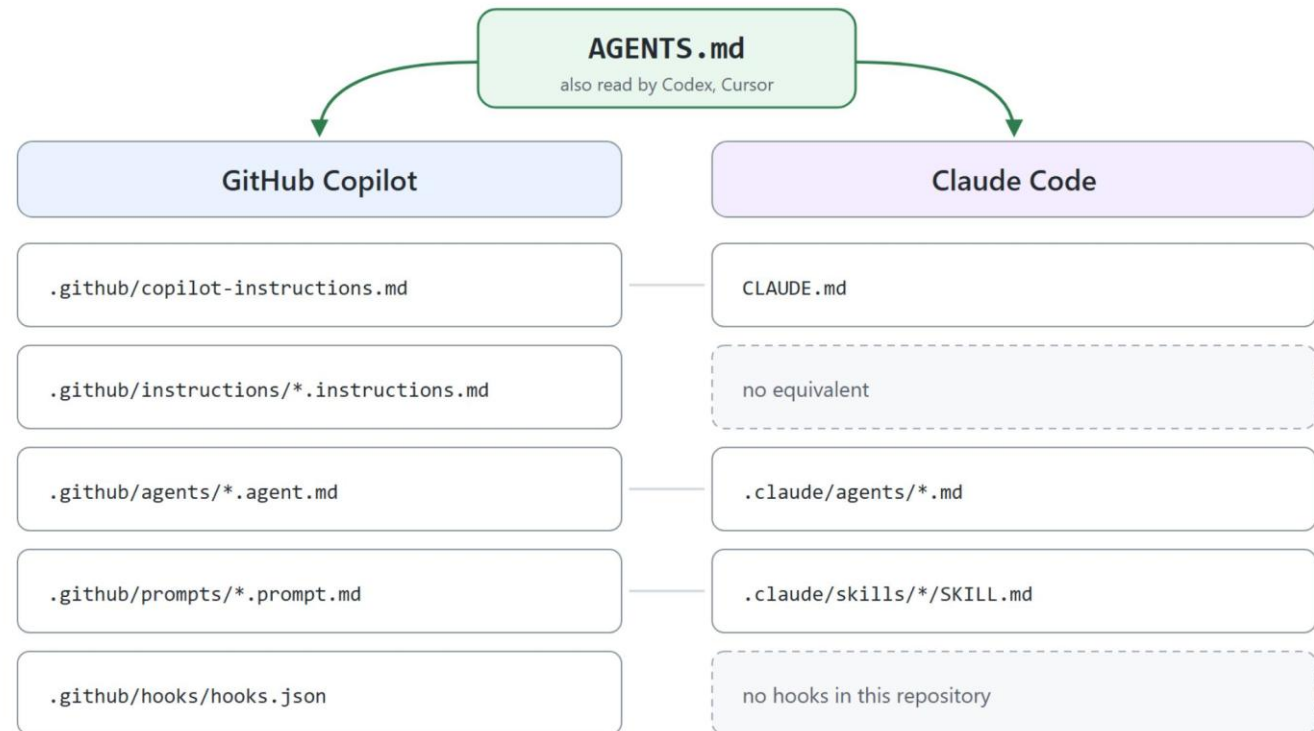
- **Sandbox:** what the agent process can touch at all; opt-in, off by default since VS Code 1.135
- **Assisted approvals:** a model auto-approves low-risk tool calls (`chat.assistedPermissions.enabled`, 1.130)
- **Hooks:** the policy only you can express, your conventions, audit trail, and integrations



One repository, two harnesses

AGENTS.md is the rule you write once

- Each harness reads **different filenames for the same job**: a mapping, not a merge
- **AGENTS.md**: plain Markdown, always on, read by Copilot, Claude Code, Codex, Cursor
- The nearest **AGENTS.md** wins; enable with `chat.useAgentsMdFile`
- A rule in both `copilot-instructions.md` and `CLAUDE.md` is **paid twice** on every turn



Where a rule belongs

Pick the layer, then pay only for what you use

- True for everyone, any harness: **AGENTS.md**
- Scoped to one stack: **.instructions.md** with **applyTo**
- A procedure a colleague would walk you through: **skill**
- Output that must be exact: **skill with a script**
- Must be enforced or audited: **hook**

